Reproducibility report - Towards Building an Intelligent Anti-Malware System: A Deep Learning Approach using Support Vector Machine (SVM) for Malware Classification [1]

SeyedBehnam Mashari **Seyed Yazdan Zinati** **Mohammadhadi Foroutan**

Reproducibility Summary

Scope of Reproducibility

We investigated the claim of paper that has proposed an AI-enabled anti-malware system which applies a Deep Learning (DL)–based method to discover the newly-generated malware [1, 2]. We used Malimg dataset comprising visualized malware binaries and has utilized this dataset to train DL-SVM models for classifying each malware family. Trained models are then loaded to give predicted accuracy for different DL-SVM models including CNN-SVM, GRU-SVM and MLP-SVM. Furthermore, confusion matrix and computational cost for these models were calculated, plotted and compared.

Methodology

We deployed Google TensorFlow for implementing deep learning algorithms as well as matplotlib, numpy and scikit-learn for computational works. We utilized dataset that is converted malware binaries into 2-D 32×32 images where class labels are malware families. After data preprocessing including re-framing and standardization dataset, models were implemented with the help of Google Tensorflow. As soon as models were trained and stored, multinomial classification of test data, plotting and visualization of predicted accuracies, F1score and confusion matrix were conducted.

Results

It is reported in the reference paper that average training accuracy for CNN-SVM, GRU-SVM and MLP-SVM are 80.96875%, 90.9375% and 99.5768229% respectively. However, tuning some of the hyperparameters resulted in better accuracy for CNN-SVM and GRU-SVM models based on our modifications. This includes changing learning rate, C in the SVM optimization problem and number of hidden layers. It will be further discussed later in the report.

What was easy

The code was not computationally expensive neither for training models nor running the classifier tasks. Furthermore, the models' codes were well-separated in different classes, enabling us to debug each part distinctively without to be concerned about affecting other parts of the code.

What was difficult

As we expected, running the code was challenging primarily because it contained packages such as TensorFlow and NumPy with some deprecated built-in functions and class methods. At the first step we tried to tackle the errors by replacing the deprecated functions with the more recent ones where applicable. Otherwise, we utilized older methods such as "compat" to access the previously defined methods inside the old versions of mentioned packages. In some cases during the training of GRU-SVM and MLP-SVM models, such built-in functions of TensorFlow were used as contrib that are actually breaking changes and their functionality has been either migrated to the core TensorFlow API.

33rd Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

Communication with original authors

We sent emails to the selected paper's author at his email address, stating some discrepancies in results after running their code and asking about their process of hyperparameter tuning.

1 Introduction

Visualization-based techniques for detecting malware, work on the principle of converting binary information of malware application files into their images. This approach enables data analysts to investigate through these binary images without executing them. However, a malware binary goes through a process to be converted from a form of 8-bit unsigned integer into classified images. A conventional process is mentioned in [3]. Referring to the article, the primary phase in the process is transforming of malware applications into images [3].

In the selected paper, the author has utilized Malimg dataset to train different deep learning using support vector machines models namely CNN-SVM, GRU-SVM, and MLP-SVM to classify malware images -which are obtained from malware binaries- into 25 predefined families such as Trojan, worm and PWS. Among all of the three mentioned models, GRU-SVM outperforms in terms of predictive accuracy (82%) while it is computationally the most expensive model compared to the other two models in the training phase -accomplished in roughly 12 minutes. It is noteworthy that Hyper-parameters such as batch size, cell-size, number of hidden layers, dropout rate, epochs and learning rate were tuned by randomly selection in the three implemented models.

2 Scope of reproducibility

One of the most prominent challenges in the information security community has been diminishing of malware. Designing a system that can detect and restrain an unknown malware is a fertile activity that should be considered and can several advantages in different sectors. Tracking and catching an unknown malware is an arduous task to tackle and may be achieved by continuously upgrading the anti-malware signature database. In fact, this database comprises the data about all identified malware found by specific identifier and then can be utilized as a reference for detecting malware. Therefore, when some new malware released, since they do not exist in the database, the system cannot detect them.

There are several claims that author states in the selected paper with respect to the results obtained in section 3. These claims are listed below:

- Claim 1. When comparing different DL-SVM models, malware families with larger number of variants score a better precision than least frequent ones in the training set. For instance Allaple A and Allaple L were the most frequent malware families in our dataset. With regards to the color-scaled confusion matrix in the original paper, the largest number of predicted labels equal to the actual ones belong to the Allaple A and Allaple L malware families, resulting in larger number of true positives and in turn a better precision.

- Claim 2. Among these three DL-SVM algorithms, GRU-SVM has the highest accuracy and f1-score on multinomial classification of malware families whereas MLP-SVM is computationally less expensive than others. This can clearly be seen in the original paper's Figure 2 where training accuracy of the three DL-SVM models on malware classification using Malimg dataset are compared. Author claims that GRU-SVM has the strongest predictive performance among others mainly due to its complex architecture that stems from its 5-layer design, potentiality of learning from sequential data, and gating mechanism given in the equation mentioned in the section 2.4.3 of [1]. However this nonlinear equations lead to relatively longer computing time in GRU-SVM model, making it the slowest DL-SVM model in this research. On the other hand, since ML-SVM model utilizes the least non-linear activation function (eg. LeakyReLU) shortest computing time is expected. The results are consistent with the hypothesis that links complexity of model's architecture design with its predictive accuracy.

- Claim 3. Finally the author's hypothesis is proposed in response to the question that why one of the models take a much longer time to be trained whereas for the other one few seconds are sufficient to be trained. It is worth mentioning that results prove the claim of author where relation between higher precision of the DL-SVM models and the number of variants belong to each malware family is investigated.

3 Methodology

The paper's author has applied some of the most famous libraries in Python. He has used Google TensorFlow for implementing deep learning algorithms. Some other Python's computing libraries such as matplotlib, numpy and scikit-learn are have been deployed to facilitate computations. Dataset is a converted malware binaries into 2-D 32×32 images and the class labels are malware families such as Worms and Trojans. Data prepossessing was done to re-frame and standardize dataset using Scikit-learn library methods such as StandardScaler(). Existing models of reputable packages were not used. However author has used a vast range of Google Tensorflow functions to train and classify models. In other words, train functions of DL-SVM algorithms were written from scratch and were bundled in specific classes to be used in the classifier using predefined Tensorflow, numpy, matplotlib and Scikit-learn .

As soon as models were trained and stored in separate folders, multinomial classification of test data can be done through each uploaded trained models. Then the results are stored for plotting and visualization of performance metrics such as predicted accuracies, F1score and confusion matrix of different DL-SVM models namely CNN-SVM, GRU-SVM and MLP-SVM.

3.1 Model descriptions

In this research work support vector machine has been used with three Deep learning methods to classify malware images. Below is a brief description to the classification algorithms that are used.

Support Vector Machine (SVM): It is used for binary classification. Discovering an optimal hyper line $f(w, x) = wx+b$ in order to segregate two classes in a given dataset is the main goal of the algorithm. To find the parameters w and b, we require to solve constrained optimization problem. L2-SVM is equivalent of L1-SVM but with the more stability in results. Author has used SVM for binary classification, however it can be also used for multinomial classification. One of the most important approaches is converting a linear model to a nonlinear model using kernel tricks.

Convolutional Neural Network (CNNs): Convolutional Neural Networks resemble feedforward neural networks. They are also comprised of hidden layers and some parameters that can be changed during the model is trained. The neurons in hidden layers receive the inputs and after execution of a dot product, the result follows by activation function that would be sigmoid, ReLU, LeakyReLU, tanh, etc. The whole process is a mapping between raw image pixels xR and class scores y, $f : xy$.

Multilayer Perceptron: A Perceptron is usually modeled by a linear function. Regarding to improve accuracy rate we should add non-linearity to the linear function by passing through an activation function such as sigmoid, tanh, ReLU, etc. Indeed, multilayer perceptron (MLP) is a kind of neural network consisting of hidden layers in which there are some perceptron. The author has used Leaky ReLU as the activation function in the investigated paper. Using the activation function, learning parameters including weight and bias for each deep learning model trained by the L2-SVM were achieved. Next, the computed loss was minimized. There is a decision function $f(x) = sign(wx + b)$ that its task is generating a vector consisting the scores for each malware family.

3.2 Datasets

To transform malware binaries into gray scale images, different methodologies are deployed. However, what is often the procedure is illustrated in [4], where the malware's binary files including various sections such as ".text" ".data" and ".rsrc" result in various images associated with different malware families. In the above-mentioned images, ".text" contains the malware's executable code, ".data" section encompasses both uninitialized code and initialized data, and ".rsrc" includes all the module resources (it may include icons, logos, or images that malware uses). In figure 1, different parts of "Trojan downloader: Dontovo. A" as a known malware are shown. These sections as 2D arrays can be visualized as distinctive image textures in the range where each pixel intensity is a number from 0 to 255.

Malimg dataset contains more than 9k malware samples from 25 different malware families including different types of worms, trojans, trojan downloaders, backdoors, password stealers (PWS) and dialers. Initially downsampled images to the fixed size of 32×32, were flattened so that a 9339×1024 size array was resulted. Whereas 70 per cent of data (6537 samples) is considered as training dataset, the rest (2802 samples) is assigned as test set. On the other hand, each family is labeled from 0 to 25 in the labels array, in the sense that the first 122 labels are 1 in column 0, showing that first 122 of variants are from "Dialer" class, the next 116 labels are 1 in column 1, showing that the next 116 number of variants are from "Backdoor" family and so on. Scikit-Learn standard scaler is utilized as the main preprocessing stage where samples' mean is subtracted from each value and then scaled to unit variance by dividing them into the standard deviation of sample values.

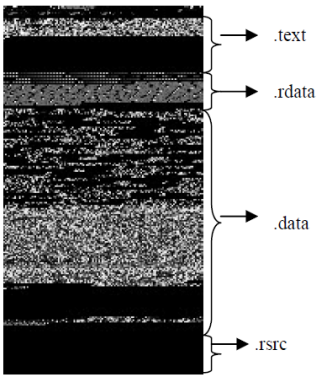


Figure 1: Different parts of a Trojan [4]

No.	Family	Family Name	No. of Variants
01	Dialer	Adialer.C	122
02	Backdoor	Agent.FYI	116
03	Worm	Allaple.A	2949
04	Worm	Allaple.L	1591
05	Trojan	Alueron.gen!J	198
06	Worm:AutoIT	Autorun.K	106
07	Trojan	C2Lop.P	146
08	Trojan	C2Lop.gen!G	200
09	Dialer	Dialplatform.B	177
10	Trojan Downloader	Dontovo.A	162
11	Rogue	Fakerean	381
12	Dialer	Instantaccess	431
13	PWS	Lolyda.AA 1	213
14	PWS	Lolyda.AA 2	184
15	PWS	Lolyda.AA 3	123
16	PWS	Lolyda.AT	159
17	Trojan	Malex.gen!J	136
18	Trojan Downloader	Obfuscator.AD	142
19	Backdoor	Rbot!gen	158
20	Trojan	Skintrim.N	80
21	Trojan Downloader	Swizzor.gen!E	128
22	Trojan Downloader	Swizzor.gen!I	132
23	Worm	VB.AT	408
24	Trojan Downloader	Wintrim.BX	97
25	Worm	Yuner.A	800

Figure 2: Malware families in the Malimg dataset [1]

3.3 Hyperparameters

Values are set manually and randomly in the code since author has directly entered hyperparameter values as the input to the methods representing different DL-SVM models. For example batch size and cell size have both been considered 256 and the learning rate and number of layers are set 1e-3 and 5 respectively. However the best hyperparameters found for every DL-SVM model are listed in figure 3.

3.4 Experimental setup and code

Code setup includes passing through specific parameters for each ".py" file as configuration. In each of these configurations some parameters are defined such as selected trained models. In fact a corresponding number to each of three available models namely CNN-SVM, GRU-SVM and MLP-SVM should be passed. Number of epochs and result path are defined in this configuration setup. Penalty parameter is another requirement for configuration setup that is an arbitrary value and is needed to solve the SVM optimization problem. What's more dataset path is defined where our Malimg dataset is located while the results are stored in a new folder. For example to run classifier.py below configuration should be set: "–model 1 –model_path ./trained-cnn-svm/ –dataset/malimg.npz". This gives average predicted accuracy using trained CNN-SVM model using test set from malimg dataset for different epochs.

Figure 3: Changing hyperparameters affected average training accuracy - red values are modified hyperparameters

Hyper-parameters	CNN-SVM		GRU-SVM		MLP-SVM	
Batch Size	256		256		256	
Cell Size	N/A		[256 × 5]		[512, 256, 128]	
No. of Hidden Layers	2		5		3	5
Dropout Rate	0.85		0.85		None	
Epochs	100		100		100	
Learning Rate	1e-3	1e-4	1e-3	1e-4	1e-3	
SVM-C	10	12	10	12	0.5	0.25

3.5 Computational requirements

Paper code was executed on a laptop with Intel Core(TM) i5-6300HQ CPU @ 2.30GHz x 4, 16GB of DDR3 RAM, and NVIDIA GeForce GTX 960M 4GB DDR5 GPU. However we conducted the reproduced code on a laptop with Intel(R) Core(TM) i5-10210U CPU @ 1.6GHz , 12GB of RAM which led to a subtle shorter time compared to the reference paper's computations speed. Generally, due to nonlinear computations and activation functions in the CNN-SVM and GNU-SVM models, longer computation time is expected in these algorithms.

It is reported that in the reference paper's laptop, training of CNN-SVM model was accomplished in around 4 minutes with average training accuracy of about 81 percent, training of GNU-SVM model was conducted in around 12 minutes with an average accuracy within epochs of 91 per cent. However training of MLP-SVM model was accomplished in only 12 seconds.

Despite what is mentioned above, In our work, these training time periods were a bit shorter than those in the reference paper. CNN-SVM model was trained in 2 minutes and 34 seconds and it was done at the step 249. Predicted accuracy was calculated fast because heavy computations were not required and the predicted labels only had to be compared to the actual ones in the test data.

Include a description of the hardware used, such as the GPU or CPU the experiments were run on. For each model, include a measure of the average runtime (e.g. average time to predict labels for a given validation set with a particular batch size). For each experiment, include the total computational requirements (e.g. the total GPU hours spent). (Note: you'll likely have to record this as you run your experiments, so it's better to think about it ahead of time). Generally, consider the perspective of a reader who wants to use the approach described in the paper — list what they would find useful.

4 Results

The hypothesis that GRU-SVM has the strongest predictive performance among others stems from its 5-layer design and gating mechanism given. It can be justified that nonlinearity in the GRU-SVM equation lead to relatively longer computing time, making it the slowest DL-SVM model among others. On the other hand, since ML-SVM model utilizes the least non-linear activation function, LeakyRELU, shortest computing time is achieved. The results demonstrate a link between complexity of model's architecture design with its predictive accuracy that seems reasonable from the author's explanations [5].

4.1 Results reproducing original paper

As the first step of reproducibilty phase we debugged, modified and executed paper code that was shared in the related repository. The first conflict that we witnessed was the average training accuracy of DL-SVM models with the reported average training accuracy by author. Moreover trained models were first executed by default hardcoded hyperparameters. To optimize the results we modified hyperparameters such as learning rate, number of layers and c in optimizing equation of SVM algorithm to compare the outcomes when passing default hyperparamters. We observed that average training accruacy was improved by modifying hyperparameters used in the original paper's code. One hypothesis that was proposed by the author was the reason behind GRU-SVM as a computationally expensive model, in the sense that non-linearity of activation function in such algorithms plays a decisive role in increasing model's complexity.

We observed that GRU-SVM model had the highest executing time among other trained models despite modifying hyperparameters. This can be justified by the initial hypothesis proposed by the author and is consistent with his claim.

4.2 Results beyond original paper

5 Discussion

Author of the original paper talks about the high precision of DL-SVM models for specific malware families such as Allaple A and Allaple L with respect to the large number of instances (variants) with these labels in the existing malware families. In our opinion, it is a strength point where the results are linked to the distribution of variants in the dataset where biases should be excluded from prejudicial conculsions. However there are several disadvantages that can be considered as validity treat to the author's work. First and foremost is the unequal number of variants in the training set which can result in an unbalanced training. Finally in the origin work, hyperparamneter tuning could be done through an iteration in the code instead of hardcoding.

5.1 What was easy

As the primary measure, we executed the code itself with respect to the required configurations. During this we entered the configuration settings into our editor's (Pycharm) config section as it was explained in the paper's related page in Github. We used the code published by the original author in GitHub to run this reproducibility study [6]. In our selected paper, parameters were defined to be passed through different python files and within different classes. For example, in main.py, one of the three models (1) CNN-SVM, (2) GRU-SVM, (3) MLP-SVM could be selected with their corresponding number. The dataset path, number of epochs and result path as well as the path to save the trained model were selected. Trained models then could be loaded for the classification purpose. The stage of passing hyperparameters was a double-edge sword . On the one hand it made code less hard coded and changing some values did not require going through the lines of codes. On the other hand it made executing code more error-prone because setting parameters required some predefined types such as –model to choose a DL-model.

The code execution for the training phase was not very heavy and for all the epochs it took few minutes to be executed.To be more specific, whereas it took less than 10 minutes for GRU-SVM model to be trained with the desired training set, CNN-SVM and accomplished its training in less than 3 minutes and MLP-SVM did so in few seconds.

5.2 What was difficult

As it was expected, running the code was challenging primarily because it contained packages such as TensorFlow and NumPy with some deprecated built-in functions and class methods. Some TensorFlow and NumPy built-ins do not exist in the newer versions of the package anymore. At the first step we tried to tackle the errors by replacing the deprecated functions with the more recent ones where applicable. Otherwise, we utilized older methods such as "compat" to access the previously defined methods inside the old versions of mentioned packages. In some cases during the training of GRU-SVM and MLP-SVM models, such built-in functions of TensorFlow were used as contrib that are actually breaking changes and their functionality has been migrated to the core TensorFlow API.

5.3 Communication with original authors

We tried to communicate with the paper author to make sure that the reproducibility report is a reasonable assessment of the selected research work. However, despite the emails and reminders that we sent during the process of code investigation, no reply was received.

In figures shared in the submitted project repository, some of these emails are captured as our effort to communicate with the paper's author. We observed some differences in obtained results after running the shared code. For instance a conflict between the obtained average training accuracy after executing the debugged code and what they claim in the paper as the average accuracy in the section "Results". Furthermore they have directly entered hyperparameter values as the input to the methods representing different DL-SVM models. For example Batch size and cell size have both been considered 256 with the learning rate of 1e-3 and 5 layers. We asked about the process of hyper-parameter tuning that has not been mentioned neither in the paper's available code in Github nor the paper passage. At any rate by investigating code it seems hyperparameter tuning has been done manualy through trying out different random values.

References

[1] Abien Fred Agarap. Towards building an intelligent anti-malware system: a deep learning approach using support vector machine (svm) for malware classification. *arXiv preprint arXiv:1801.00318*, 2017.

[2] Daniel Gibert, Carles Mateu, Jordi Planes, and Ramon Vicens. Using convolutional neural networks for classification of malware represented as images. *Journal of Computer Virology and Hacking Techniques*, 15(1):15–28, 2019.

[3] Jaiteg Singh, Deepak Thakur, Farman Ali, Tanya Gera, and Kyung Sup Kwak. Deep feature extraction and classification of android malware images. *Sensors*, 20(24):7013, 2020.

[4] Lakshmanan Nataraj, Sreejith Karthikeyan, Gregoire Jacob, and Bangalore S Manjunath. Malware images: visualization and automatic classification. In *Proceedings of the 8th international symposium on visualization for cyber security*, pages 1–7, 2011.

[5] Mahmoud Kalash, Mrigank Rochan, Noman Mohammed, Neil DB Bruce, Yang Wang, and Farkhund Iqbal. Malware classification with deep convolutional neural networks. In *2018 9th IFIP international conference on new technologies, mobility and security (NTMS)*, pages 1–5. IEEE, 2018.

[6] Abien Fred Agarap. Afagarap/malware-classification v0.1-alpha, December 2017.